

How Can Quantum Computing and Databases Meet “in Practice”? From Algorithms to Systems

**Hanwen Liu¹, Yiran Li², Ibrahim Sabek¹,
Xuanhe Zhou³, Hans-Arno Jacobsen²**

¹University of Southern California

²University of Toronto

³Shanghai Jiao Tong University



Our physical world is quantum

At microscopic scales, nature behaves differently.

Quantum systems can exhibit **wave-like characteristics**, become **entangled**, and yield probabilistic outcomes when **measured**.



Schrödinger's cat

Schrödinger imagined a cat whose fate is controlled by a quantum event.

In the thought experiment, until that event is observed, the cat is described as **both alive and dead**.

A quantum computer encodes information in quantum states, controls how those states evolve, and measures them to obtain a classical outcome.

Quantum x Database: an unlikely connection?

Not really.

Database was there from the beginning!

“A fast quantum mechanical algorithm for database search”

Lov Grover, 1996

Grover's algorithm gives a quadratic query advantage for **unstructured search**, which is a natural primitive for retrieval.

Several database primitives have quantum algorithmic counterparts

Unstructured search

Selection, lookup, multi-item retrieval

→ Grover search / amplitude amplification

Discrete combinatorial optimization

join ordering, index selection, transaction scheduling

→ QAOA, quantum annealing, other quantum optimization methods

The algorithmic connection between quantum computing and database systems is clear; the challenge is turning it into practical database systems.

Tutorial outline

3 hours • 6 parts • 2 hands-on demonstrations

01 Quantum Background & Motivation

20 minutes

02 Quantum Computing Paradigms & Algorithms

35 minutes

03 Quantum Computing for Database Systems

65 minutes

04 Data Management for Quantum Computing

10 minutes

05 QC & DB Meet “in Practice”

30 minutes

06 Open Challenges & Research Opportunities

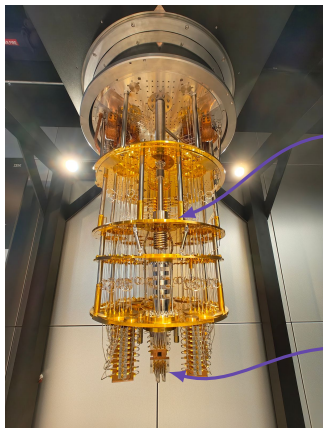
15 minutes

01

PART 1 OF 6

Quantum Background & Motivation

Where does a quantum program actually run?



IBM Quantum System One, 2019.

QUANTUM SYSTEM

Quantum processor Physical qubits where quantum operations execute

Quantum computer QPU + cryogenics + classical hardware for control, calibration, and scheduling

CLASSICAL ALTERNATIVE

Simulator Runs quantum circuits on classical hardware for testing and small-scale experiments

From bits to qubits

Classical bit

$$b \in \{0, 1\}$$



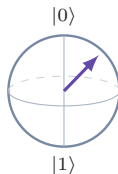
At any moment, the bit is **0 or 1**.

Qubit

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}$$
$$|\alpha|^2 + |\beta|^2 = 1$$

A qubit state is a superposition of the basis states $|0\rangle$ and $|1\rangle$.

α and β are **amplitudes**: complex coefficients that specify the contribution of $|0\rangle$ and $|1\rangle$ to the state.



Bloch sphere

Measurement still returns one classical bit: 0 or 1.

Amplitudes carry both magnitude and phase

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \alpha, \beta \in \mathbb{C}$$

Magnitude $\rightarrow$ probability

$$P(0) = |\alpha|^2, P(1) = |\beta|^2$$

$$|\psi\rangle = \sqrt{0.7}|0\rangle + \sqrt{0.3}|1\rangle$$

Outcome 0 70%

Outcome 1 30%

The magnitude of an amplitude determines measurement probability.

Phase $\rightarrow$ interference

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Both give $P(0) = P(1) = 50\%$.

But when amplitudes combine, relative phase determines the result.

$$a + a = 2a$$

reinforce

$$a - a = 0$$

cancel

Interference can reinforce some outcomes and suppress others.

Computation is controlled evolution of quantum states

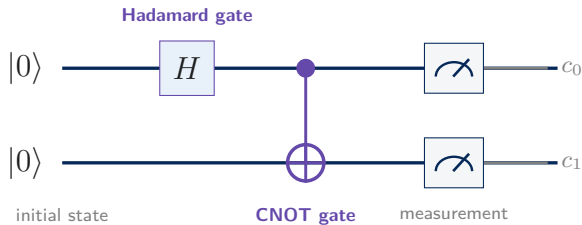
Quantum gates

apply unitary transformations to one or more qubits that are reversible before measurement.

X flips $|0\rangle \leftrightarrow |1\rangle$.

H creates state superposition.

CNOT is the controlled-NOT gate that couples two qubits.



A quantum circuit is the quantum analogue of a program.

Creating superposition with a quantum circuit

Hadamard gate mixes $|0\rangle$ and $|1\rangle$ with equal magnitude.

$$|0\rangle \xrightarrow{H} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Measurement gives **0 or 1**, each with probability 50%.

Construct it with Qiskit

Qiskit is IBM's open-source SDK for quantum computing, from circuit construction to execution.

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)
```

QuantumCircuit(1, 1): 1 qubit + 1 classical bit.

A Hadamard gate transforms the basis state $|0\rangle$ into an equal superposition.

Creating entanglement with a quantum circuit

Two qubits have joint basis states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

Hadamard creates superposition; CNOT correlates the qubits.



The two qubits cannot be described independently.
Measurement gives **00 or 11**, each with probability 50%.

Construct it with Qiskit

The same library constructs multi-qubit gates and measurements.

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])
```

`cx(0, 1)`: control qubit 0, target qubit 1.

Hadamard followed by CNOT creates a Bell state with correlated measurement outcomes.

Noise makes quantum computation unreliable

Physical gate errors today

Single-qubit gate

0.01–0.1%

about 1 in 10^3 – 10^4

Two-qubit gate

$\sim 0.1\%$

about 1 in 10^3

Representative superconducting-QPU values; rates vary by device, qubit, coupling, and calibration.

Where errors enter

- **Decoherence:** quantum information degrades while the computation runs
- **Imperfect gates:** physical operations can deviate from the intended gates
- **Limited connectivity:** routing may require extra two-qubit gates
- **Readout error:** measurement can report the wrong bit

As circuits get deeper, small gate errors compound and error rate increases.

From NISQ to fault-tolerant quantum computing

NISQ: today

$10^2 - 10^3$ physical qubits

- Noisy Intermediate-Scale Quantum
- Noisy gates and measurements
- Shallow circuits; error mitigation rather than full correction



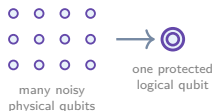
Error correction

- Common approach: encode one **logical qubit** across many physical qubits
- Detect and correct errors repeatedly
- Substantial hardware and decoding overhead



Fault tolerance

- Logical errors far below physical errors
- Long, reliable computations
- Deeper simulation, factoring, and optimization



Vendor targets for fault-tolerant QC

IBM Starling: **2029**

Quantinuum Apollo: **2029**

IonQ FT system: **2030**

Targets are company projections.

What a quantum computer is—and is not

IT IS

- Physical system whose state is described by **qubits**
- Controlled through quantum operations and read through **measurement**
- Computing model whose algorithms can scale differently from classical ones

IT IS NOT

- Universally **faster CPU**
- Machine that tries every answer and lets us **read them all**
- Replacement for classical computers

Can a classical computer simulate a quantum computer?

Yes, in principle. But general quantum circuits can require exponential classical resources.

02

PART 2 OF 6

Quantum Computing Paradigms & Algorithms

Quantum Computing Paradigms

Annealing

Use a specialized quantum annealer that evolves a physical system toward low-energy states encoding solutions to Ising or QUBO problems.

Gate-based

Apply sequences of quantum gates to qubits to manipulate their amplitudes and correlations. This is the universal circuit model for quantum algorithms.

Hybrid

Combine quantum methods with classical orchestration, optimization, and post-processing.

Quantum-inspired

Adapt ideas from quantum algorithms or quantum physics into methods that run entirely on classical computers or specialized classical accelerators.

These paradigms solve problems differently and make different demands on hardware, algorithms, and workflow.

A common language for quantum optimization: QUBO and Ising

Binary optimization: QUBO

Quadratic Unconstrained Binary Optimization

$$\min_{x \in \{0,1\}^n} \left(\sum_i q_i x_i + \sum_{i < j} q_{ij} x_i x_j \right)$$

x_i : binary decisions (0 or 1)

linear terms: individual costs

quadratic terms: pairwise interactions

Many combinatorial problems fit this form, with constraints commonly converted to penalties.

Variable transform

$$z_i = 1 - 2x_i$$

$$x_i \in \{0, 1\}$$



$$z_i \in \{-1, +1\}$$



same objective,
different variables

Energy / Hamiltonian form: Ising model

$$\min_{z_i \in \{-1, +1\}} \left(\sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j \right)$$

z_i : spin variables (-1 or +1)

h_i : preference for one spin value

J_{ij} : pairwise interaction between spins

The objective assigns an energy to every spin configuration.

Quantum annealing searches this energy landscape for low-energy states.

QUBO and Ising encode the same optimization landscape in binary and spin variables.

Quantum annealing: from QUBO to a low-energy solution

Encode the optimization objective as an Ising Hamiltonian, then evolve the system toward its low-energy states.

1. Encode the problem

$$H_P = \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j$$

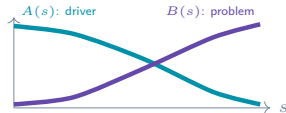
H_P : problem Hamiltonian
lower energy $\leftrightarrow$ better solution

The QUBO is converted into this equivalent Ising Hamiltonian.

2. Anneal from an easy Hamiltonian to the problem Hamiltonian

$$H(s) = A(s)H_D + B(s)H_P$$

H_D is the driver Hamiltonian; $s \in [0, 1]$ is normalized annealing time.



Start: $A(s) \gg B(s)$; prepare the easy ground state of H_D .

End: $A(s) \ll B(s)$; the problem Hamiltonian H_P dominates.

3. Why this can work

Adiabatic evolution

Start in the ground state of H_D



Change $A(s), B(s)$ sufficiently slowly



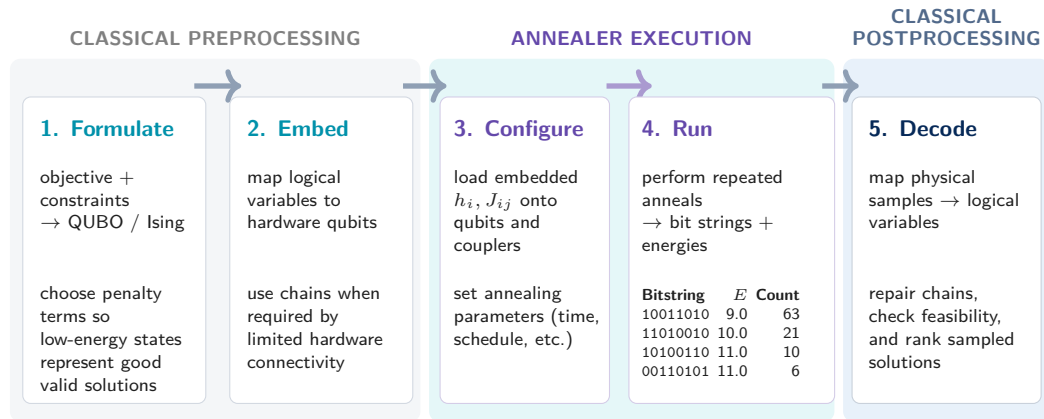
Stay near the instantaneous ground state

Final state $\approx$ ground state of H_P

which encodes a minimum of the optimization objective.

In practice, noise, thermal effects, finite runtime, and limited connectivity mean annealers seek low-energy candidates rather than guaranteeing the optimum.

Quantum annealing in practice: formulate, embed, anneal, decode



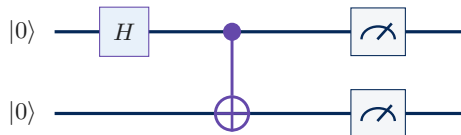
In practice, users formulate the problem as a QUBO; software stack handles hardware-specific mapping.

Gate-based quantum computing: build, transpile, execute, measure

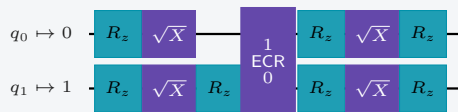
Workflow

1. Build a logical circuit
2. Transpile to device-native gates and connectivity
3. Execute repeatedly on a QPU or simulator
4. Interpret measurement results as classical samples or estimates

Logical circuit

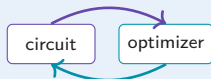


Example transpiled circuit



A logical circuit is transpiled into a hardware-compatible circuit, then executed repeatedly to produce classical measurement results.

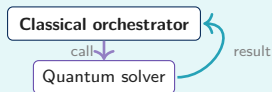
Hybrid quantum–classical workflows



Variational algorithms

- Quantum circuit evaluates a parameterized objective
- Classical optimizer updates the parameters

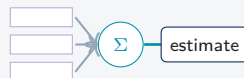
Examples: Quantum Approximate Optimization Algorithm (QAOA); Variational Quantum Eigensolver (VQE)



Hybrid solvers

- Classical orchestrator manages the workflow
- Quantum solver is called repeatedly

Example: D-Wave hybrid solvers



Error mitigation

- Run circuits under modified conditions
- Combine results classically to reduce noise bias

Examples: zero-noise extrapolation; probabilistic error cancellation

QAOA: a hybrid gate-based algorithm for combinatorial optimization

Quantum Approximate Optimization Algorithm

QAOA alternates cost and mixer evolutions to steer a parameterized state toward low-cost solutions.

H_M : mixer Hamiltonian that explores solutions; H_C : cost Hamiltonian that encodes the objective.



1. Initial state

$$|\psi_0\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle$$

Begin in an equal superposition over all 2^n candidate bit strings.

2. Cost operator

$$U_C(\gamma) = e^{-i\gamma H_C}$$

In this unitary operator, the objective Hamiltonian assigns cost-dependent phases.

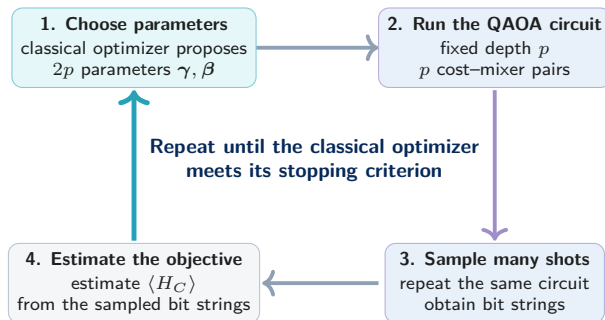
3. Mixer operator

$$U_M(\beta) = e^{-i\beta H_M}$$

Mixing and interference turn phase differences into changes in sampling probability.

The best γ, β are problem-dependent and unknown in advance, so a classical optimizer tunes them.

QAOA tunes a parameterized quantum circuit in a classical loop



Depth p

- p cost-mixer layer pairs
- $2p$ variational parameters
- Larger p : more expressive, but deeper and harder to optimize

After optimization

Sample again using the best parameters

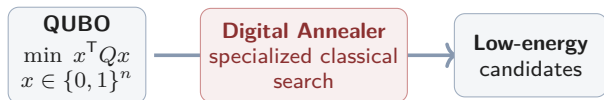
→ obtain candidate solutions

Depth p is fixed during optimization; each optimizer iteration updates the $2p$ parameters.

Quantum-inspired optimization: solving QUBO on classical hardware

Digital Annealer (e.g., Fujitsu) uses specialized classical hardware for annealing-inspired QUBO search.

How it works



evaluate many bit-flip moves in parallel
accept moves using an annealing-style rule

User view: formulate QUBO $\longrightarrow$ submit $\longrightarrow$ receive candidate solutions

Key features

- Specialized classical hardware for combinatorial optimization
- Parallel evaluation of many candidate bit flips
- Dense QUBO connectivity
- High-level QUBO interface similar to an annealing solver
- Annealing-inspired search can escape local minima

Choose a paradigm by formulation, hardware, and workflow cost

	Annealing	Gate-based	Hybrid	Quantum-inspired
Input	QUBO / Ising	quantum circuit / Hamiltonian	problem + quantum subroutine	QUBO / Ising
Execution	quantum annealer	gate-based QPU	classical-quantum hybrid workflow	specialized classical hardware
Main bottleneck	embedding, connectivity, and sampling quality	circuit depth, gate errors, and connectivity	repeated QPU calls and classical-quantum coordination	classical compute time and memory
Output	sampled candidate solutions + energies	samples or expectation estimates	iteratively improved samples / estimates	sampled candidate solutions + objective values

Compare end-to-end solution quality, latency, and resources against strong classical baselines.

A representative gate-based algorithm: Grover search

A canonical example of how quantum interference can improve search complexity.

Search problem

$$f(x) = \begin{cases} 1, & x \text{ is marked,} \\ 0, & \text{otherwise.} \end{cases}$$

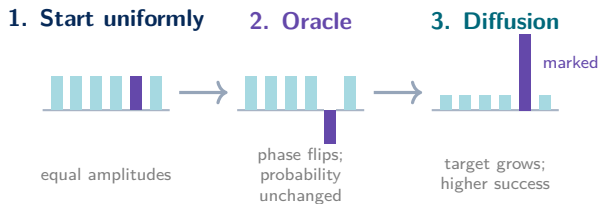
Given $N = 2^n$ candidates, find a marked item using queries to f .

Classical: $O(N)$ queries

Grover: $O(\sqrt{N})$ queries

Query complexity: assumes access to an oracle implementing f .

How amplitude amplification works



The oracle first changes only the relative phase of the marked state. Diffusion then converts that hidden phase difference into a change in amplitude, making the marked state more likely to appear when measured.

Grover search uses a reversible oracle to mark valid solutions

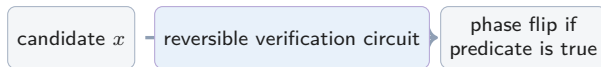
What the oracle does

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle$$

predicate	phase action
$f(x) = 0$	unchanged
$f(x) = 1$	flips sign

The oracle encodes the predicate into phase; it does not directly reveal the answer.

How to build one



- Compute the search predicate reversibly
- Use ancilla qubits for intermediate values when needed
- Uncompute temporary values after the phase mark

Practical challenge

- Oracle depth can dominate the circuit
- Reversible logic adds gates and workspace qubits
- Coherent Grover iterations likely require fault-tolerant hardware

From unstructured search to optimization: Grover Adaptive Search

Turn an objective into a search predicate

Given an optimization problem

$$\min_{x \in \{0,1\}^n} C(x)$$

Choose a current threshold y and mark any candidate that improves it:

$$f_y(x) = 1 \iff C(x) < y$$

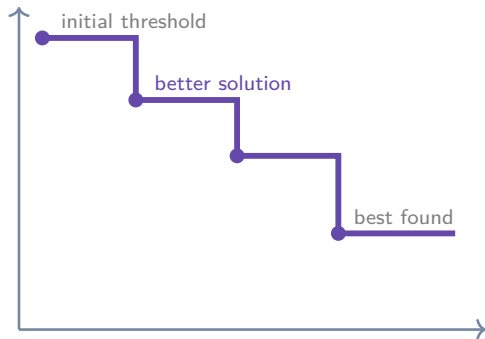
Grover search can now look for any x with $C(x) < y$.

The threshold turns “find the minimum” into repeated “find something better” searches.

Adaptive search loop

1. Start with a threshold y
2. Use Grover search to find a candidate x with $C(x) < y$
3. If found, update $y \leftarrow C(x)$
4. Repeat until the stopping criterion or evaluation budget is reached

Grover Adaptive Search finds better solutions with quadratically fewer queries



Why the threshold adapts

- Successful search lowers the threshold y
- The oracle then marks only better solutions
- Repeat until the stopping / confidence criterion is met

Global optimum with high probability

Repeated adaptive searches improve the incumbent until the optimum is found with high probability.

Success probability can be increased by repetition.

oracle-query complexity

$$O(N)$$

classical search

$$O(\sqrt{N})$$

GAS

assuming an efficiently implementable oracle

Hands-on: a basic Grover search example

Open the interactive Colab notebook and run a simulated Grover search circuit on your own device.

bit.ly/vldb-grover

Before you start: use a stable connection (for example, a mobile hotspot) rather than hotel Wi-Fi, and check the **CPU** / **MEM** runtime status at the top right.

03

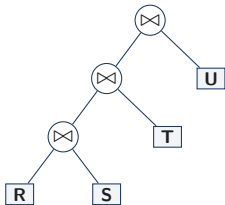
PART 3 OF 6

Query Optimization: From Formulation to Systems

Join Ordering: The Problem and Why It Is Hard

The Problem

Choose the order in which the relations of a query are joined.



One of many valid join trees.

Why It Is Hard

$n!$

Left-deep orders for n relations,
already 3,628,800 at $n = 10$

$> 10\times$

Cost spread between good and bad plans,
visible directly in **query latency**

Join order directly affects query latency.

Join Ordering: A Natural Testbed

Gate Based
Quantum Circuits

Nayak et al. '23

Quantum ML
Learned Plan
Selection

Winker et al. '23

Annealing
Quantum
Annealers

Schönberger et al. '23

**Quantum
Inspired**
Digital Annealers

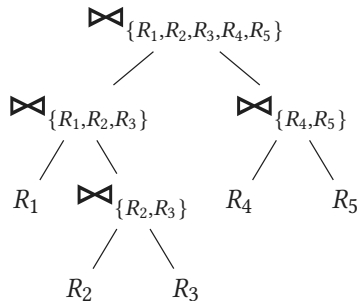
Schönberger et al. '24

Hybrid
Quantum +
Classical

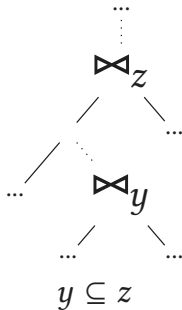
Saxena et al. '24

Five quantum paradigms tackle the same problem.

Gate Based: Bushy Join Trees



(a) Example



(b) Constraint

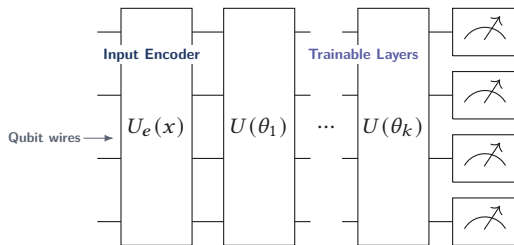
1. Represent candidate subsets. Create one binary variable x_S for every relation subset S with $|S| \geq 2$.

2. Enforce a tree. Overlapping selected subsets must be nested; disjoint subsets may become siblings. Invalid overlaps receive a large quadratic penalty.

3. Join cost. Linear weights encode estimated subset cost.

The QUBO rewards low-cost subsets and penalizes non-nested overlaps; the formulation requires $O(2^m)$ variables.

Quantum ML: Variational Circuits Learn the Join Order



Input encoder: angles come from table IDs.
Trainable layers: angles are learned from data.

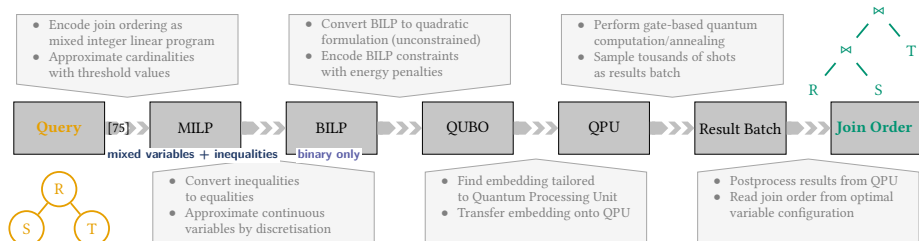
1. Encode the query. Input x lists the table IDs. For each ID j , set $\phi_j = j\pi/(n-1)$. Then $R_x(\phi_j)$ rotates one qubit around its x -axis by that angle, turning the ID into a circuit input.

2. Train the circuit. Layers $U(\theta_1), \dots, U(\theta_k)$ apply trainable rotations after input encoding. Their angles are learned from training data.

3. Measure and select. Output states represent candidate orders; measurement identifies the highest-scoring valid order.

PostgreSQL closes the loop: execution times define the training rewards.

Co-Design: Shaping the Encoding for the Hardware



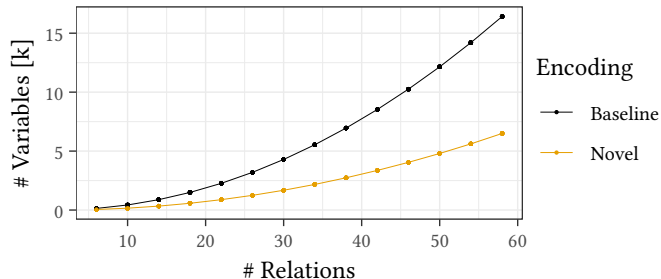
1. Formulate. MILP → BILP → QUBO.

2. Embed and sample. Map the logical QUBO to the hardware topology, run repeated shots, and return a batch of sampled assignments.

3. Decode and verify. Reject invalid samples, reconstruct join orders, and recompute their costs.

Real-hardware scope: 3 relations on IBM Q; up to 5 on a D-Wave quantum annealer.

Quantum Inspired: Digital Annealing at Scale



Baseline. At each join, separately record the new relation and all relations already in the partial plan, plus extra constraint variables.

Novel. One variable records whether relation r has joined by step j . Penalties enforce a valid join order.

The leaner QUBO reaches 50-relation joins on (classical) quantum-inspired hardware.

Formulating the Join Order Problem as a QUBO

$$E_{\text{JO}} = \underbrace{A(H_{\text{Va}} + H_{\text{Vb}} + H_p)}_{\text{Validity Penalties}} + \underbrace{H_{\text{Cost}}}_{\text{Estimated Plan Cost}}$$

QUBO: Quadratic energy over binary variables, native to annealers.

Decision Variables

$x_{rj} = 1$: Relation r at join step j

$y_{pj} = 1$: Predicate p active at step j

Validity Constraints

H_{Va} : $j + 1$ relations at step j

H_{Vb} : Selected relations stay selected

H_p : Both predicate relations are present

Large A : A constraint violation must cost more than it can save.

Inside the Encoding: A Concrete Example

$$H_{Va} = \sum_{j=1}^J \left(b_j - \sum_{r=1}^R \text{roj}_{rj} \right)^2 \quad H_{Vb} = \sum_{r=1}^R \sum_{j=2}^J \text{roj}_{r,j-1} (1 - \text{roj}_{rj})$$

Relations A, B, C and joins i, j : Step i must hold $b_i = 2$ relations, step j must hold $b_j = 3$.

Valid: $(A \bowtie B) \bowtie C$

	join i	join j
A	1	1
B	1	1
C	0	1

$$H_{Va} = (0)^2 + (0)^2 = 0$$

$$H_{Vb} = 0$$

No penalty, the annealer prefers it.

Invalid: All Three at Join i

	join i	join j
A	1	0
B	1	0
C	1	0

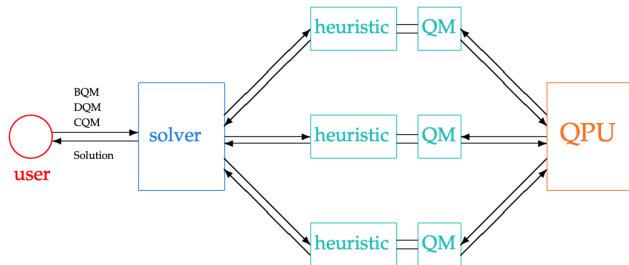
$$H_{Va} = (-1)^2 + (3)^2 = 10$$

$$H_{Vb} = 1 + 1 + 1 = 3$$

Extra energy, the annealer avoids it.

Validity is priced rather than directly checked: An invalid assignment always carries extra energy.

Hybrid Solver



1. Choose a model. Translate the DB optimization problem into **BQM** (binary decision), **DQM** (discrete choice), or **CQM** (constrained model).

2. Solve hybridly. Classical heuristic threads exchange candidates with quantum modules that create native QUBO subproblems for the QPU.

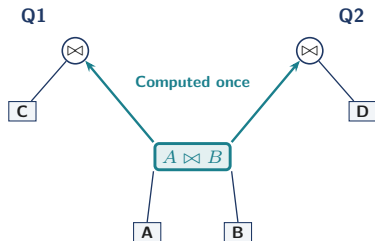
3. Return a feasible order. At the time limit, extract the best assignment satisfying the declared constraints and decode its join order and cost.

**CQM avoids manual penalty tuning (i.e., A) at the interface.
Its internal orchestration remains a black box.**

MQO: The Problem and Why It Is Hard

The Problem

Optimize a *batch* of queries together: plans can share work.



Compute the shared intermediate once, feed both plans.

Why It Is Hard

Coupled Decisions

The objective is **global**, not per-query.

Larger Search Space

Plan combinations grow with batch size.

The Paper That Started It All: MQO on a Quantum Annealer

2016

The first MQO study on a real quantum annealer

What They Ran It On

The D-Wave 2X: Early quantum annealer.

What They Found

Over 1,000× optimization-time speedup on selected synthetic instances.

MQO: Constructing the QUBO

$$E_{\text{MQO}} = w_L E_L + w_M E_M + E_C + E_S$$

E_L : Select a Plan

Rewards selecting plans.

E_M : One per Query

Penalizes multiple plans.

E_C : Plan Cost

Adds the estimated cost of each selected plan.

E_S : Sharing Savings (Negative Value)

A negative value represents pairwise savings.

The Practical Gap, Shared by Both Problems

From the Solver's Side

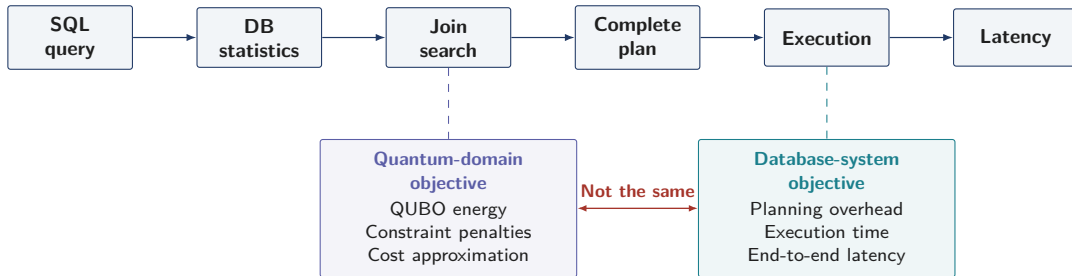
- QUBO energy is not latency
- Mapping the problem onto hardware dominates optimization latency
- Samples do not directly provide a DB solution

From the Database's Side

- Cost models must reflect DBMS behavior
- Decoded plans must be valid and executable
- Optimization latency and DB performance must be evaluated together

Formulation has been investigated for years, but control and deployment remain open.

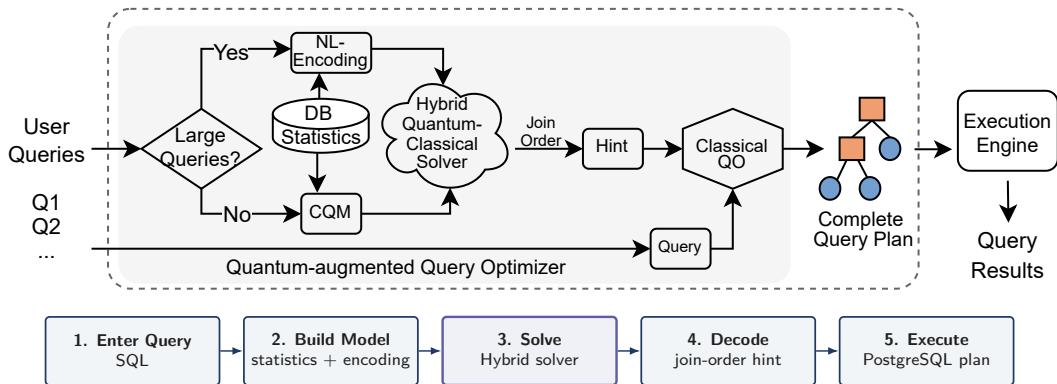
The Solver Objective Is Not Enough



Practical Evaluation Rule

Evaluate **solution quality**, **optimization latency**, **DB execution performance**, and **end-to-end latency** together.

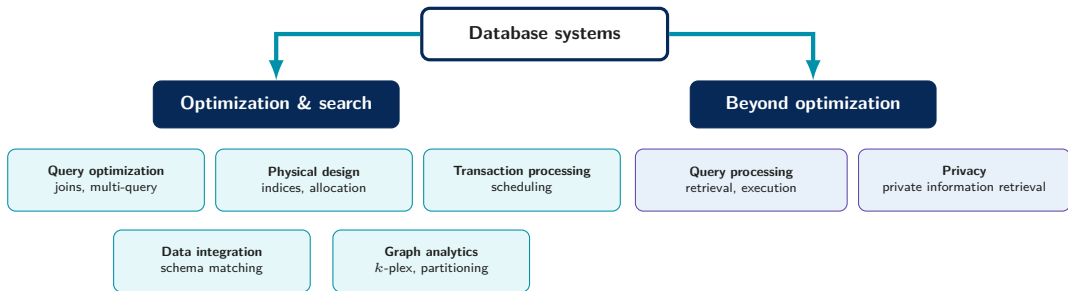
Q²O: Quantum-Augmented Join Ordering Inside PostgreSQL



LIVE DEMO

One query, from SQL to an executed PostgreSQL plan.

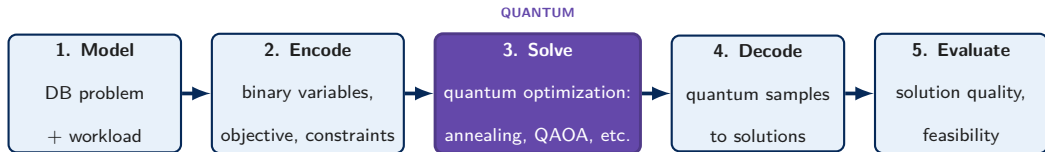
The emerging landscape of quantum computing for databases



A meeting point for two research communities

Database systems offer rich optimization, search, and analytics problems. Quantum computing opens new ways to formulate and explore their solution spaces.

A common quantum-optimization workflow



Example: graph partitioning

- Vertices $\rightarrow$ binary variables / logical qubits
- Edges $\rightarrow$ pairwise cost terms / logical couplings
- Objective $\rightarrow$ cost Hamiltonian

Where database research contributes

- Faithful objectives and constraints
- Compact encodings and decomposition
- End-to-end evaluation against strong baselines

Index selection as a quantum optimization problem

Database decision

Choose an index configuration that minimizes workload cost under storage and maintenance constraints.

The general problem is NP-hard, as each candidate index is a binary decision, and the search space grows combinatorially.

$$\min_{x \in \{0,1\}^m} \underbrace{C_{\text{workload}}(x)}_{\text{workload cost}} + \lambda \underbrace{P_{\text{budget}}(x)}_{\text{budget penalty}}$$

$x_i = 1$ means “build candidate index i ”

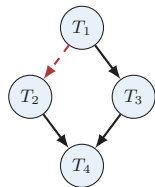
Approaches explored so far

- **Annealing and QAOA:** formulate index selection as a QUBO problem.
- **Quantum search:** use Grover search to explore index configurations.
- **QML:** learn index recommendations from workload features.

Opportunities for database research

Generating index candidates, estimating index construction and maintenance costs, and adapting to workload drift require tight integration between the quantum optimizer and the database system.

Transaction scheduling in quantum formulations



T_i : transaction

$T_i \rightarrow T_j$: dependency; T_i must precede T_j

$T_i \dashrightarrow T_j$: conflict; T_i is scheduled before T_j

Scheduling objective

Assign transactions to machines and start times while avoiding conflicts and minimizing overall completion time.

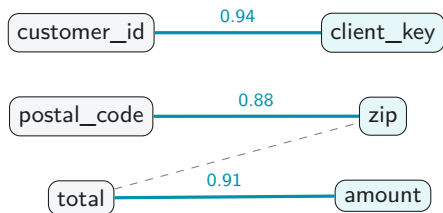
Approaches explored so far

- **Annealing/QUBO**: binary assignment of transactions to positions; penalize conflicts and invalid schedules.
- **Grover search**: an oracle marks schedules that satisfy dependencies and objective thresholds.
- **Iterative hybrid**: lock part of the schedule, solve a smaller QUBO, and repeat.

An opportunity for hybrid scheduling

Split large schedules into hardware-sized subproblems while preserving global quality and adapting to changing conflicts.

Schema matching with annealing and QAOA



Matching objective

Select high-quality attribute correspondences while satisfying cardinality and compatibility constraints.

Many practically relevant global-matching variants are NP-hard.

Quantum formulation

- One bit x_{ij} represents a candidate match.
- Linear terms reward similarity; quadratic penalties discourage incompatible combinations.
- Annealing or QAOA searches the resulting constrained space.

Beyond 1-to-1 matching

1: N and N :1 correspondences support matches between individual attributes and attribute groups, but yield a harder binary optimization problem.

Graph analytics with quantum approaches

Graph problems studied in quantum algorithm literature

Quantum algorithms have been developed for max-cut, graph coloring, maximum clique, maximum independent set, graph partitioning, Hamiltonian cycle, TSP, etc.

Maximum k -plex: search + verification

Find the largest subset where each vertex may miss edges to at most $k - 1$ other selected vertices.

- Encode a candidate vertex subset
- Check size and k -plex feasibility
- Amplify valid subsets with Grover search

Hypergraph partitioning: energy minimization

Partition vertices into balanced blocks while minimizing cut hyperedges.

- Encode block assignments
- Add cut-cost and balance terms
- Solve the QUBO directly or in smaller pieces

Potential quantum execution and retrieval - overview

Potential Quantum Benefits for DBMSs

- **Enhanced Query Performance:** Quantum algorithms may accelerate search, deduplication, etc.

- **Improved Transaction Concurrency:** Faster queries shorten transactions and reduce conflicts.

- **Efficient Rollback:** Reversible quantum operations may reduce rollback overhead.

Practical Challenges

- **Quantum constraints:** Probabilistic results, no-cloning, and limited control flow complicate reliable execution.

- **Data loading:** Encoding classical data into qubits may offset quantum speedups.

- **Hardware limitations:** Limited qubits, short coherence times, and high error rates restrict scalability.



[1] Manish Kesarwani and Jayant R. Haritsa. 2024. Is Quantum-Based SQL Query Execution Viable?. In VLDB Workshops. VLDB.org.

[2] Tim Littau, Ziyu Li, and Rihan Hai. 2024. Towards Quantum Data Structures for Enhanced Database Performance. In VLDB 2024 Workshop: The Second International Workshop on Quantum Data Science and Management (QDSM'24).

Potential quantum execution and retrieval - operators

Mapping Classical SQL Operators to Quantum Circuits

Type	Example SQL Operators	Quantum Methods	Complexity: Quantum vs. Classical
Filtering	WHERE, LIKE, EXISTS	Grover Search, Quantum Counting	$O(\sqrt{N})$ vs. $O(N)$
Join	Equi-join, non-equi join	Grover Search	$O(\sqrt{N_2})$ per probe vs. $O(N_1 \times N_2)$
Aggregation	MIN	Minimum Finding	$O(\sqrt{N})$ vs. $O(N)$

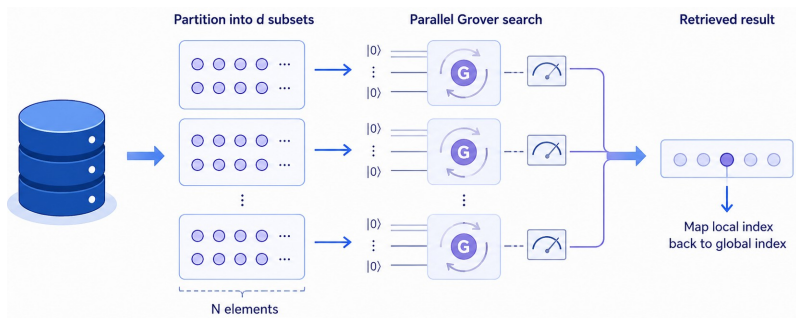
Notation. N : number of rows; N_1 and N_2 : sizes of the joined tables; ϵ : estimation precision.

[1]: Muzhi Chen, Xuanhe Zhou, Wei Zhou, Bangrui Xu, Surui Tang, Guoliang Li, Bingsheng He, Yeye He, Yitong Song, and Fan Wu. 2026. Qute: Towards Quantum-Native Database. CoRR abs/2602.14699 (2026)

Potential quantum execution and retrieval - data structures

Quantum Partitioned Database (QPD)

- **Motivation:** Reduce the circuit depth of Grover search over large databases.
- **Main idea:** Partition N elements into d subsets and search them in parallel with Grover's algorithm, reducing the ideal search depth from $O(\sqrt{N})$ to $O\left(\sqrt{\frac{N}{d}}\right)$.
- **Trade-off:** Require additional qubits, parallel hardware resources, and partitioning overhead.



Quantum-enhanced approximate query processing

AQP estimates aggregate queries from samples with bounded error, enabling interactive analytics on large datasets when exact processing is too slow.

$$\hat{N}_{P,i} = N \frac{\tilde{N}_{P,i}}{\tilde{N}}$$

$$\text{Goal: } \Pr\left(\left|\hat{N}_{P,i} - N_{P,i}\right| \leq \varepsilon\right) \geq 1 - \delta$$

N : total number of tuples

$\tilde{N}$: number of sampled tuples

P : query predicate

i : target group (e.g., Asia)

$\tilde{N}_{P,i}$: sampled tuples in group i satisfying P

Approach explored so far

The existing quantum approach uses a hybrid quantum-classical framework for rare groups in long-tail distributions:

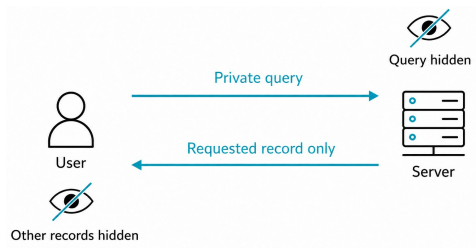
1. Classical sampling: Identify rare groups.
2. Quantum amplification: Apply amplitude amplification to increase their sampling probability.
3. Classical correction: Refine estimates to improve long-tail accuracy.

Current limitations

The scalability is limited by repeated amplification for rare groups and costly QRAM-based data loading.

Quantum privacy for databases - task definition

Database Privacy Task



Symmetric Private Information Retrieval (SPIR)

- **User privacy:** The server cannot determine which record the user requests.
- **Data privacy:** The user learns only the requested record and nothing about the others.

Why Quantum?

- **Superposition:** A query can be hidden among multiple possible requests.
- **Measurement Disturbance:** Inspecting the query alters its state, making cheating detectable.
- **No-Cloning:** Unknown quantum states cannot be perfectly copied for later analysis.

These properties enable privacy guarantees unavailable through classical communication alone.

Quantum privacy for databases - quantum approaches proposed

Approach 1 · Superposition-Based Quantum Private Query (QPQ)

1. Prepare: The user sends a direct query and a decoy superposition.

$$|q_j\rangle = |j\rangle, \quad |q_d\rangle = \frac{|j\rangle + |r\rangle}{\sqrt{2}}$$

2. Query: The server coherently accesses the database via qRAM.

3. Verify: Server measurement collapses the superposition, enabling cheating detection.



Strength: Strong database privacy with logarithmic communication and query complexity.

Limitation: Transmission loss may require repeated queries, risking privacy leakage and thus motivating a loss-tolerant QKD-based protocol (Approach 2).

Quantum privacy for databases - quantum approaches proposed

Approach 2 • QKD-Based Practical QPQ

1. Generate: Modified QKD generates an oblivious key K^f : the server knows all bits, while the user knows only a few.

$$K^f = K^{(1)} \oplus \dots \oplus K^{(k)}$$

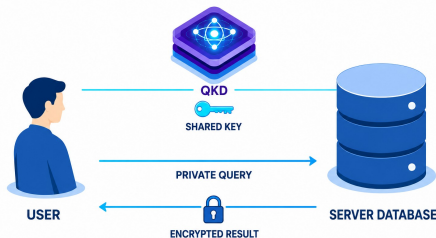
2. Align: The user shifts a known key bit K_j^f to the requested database item X_i .

$$s = j - i$$

3. Retrieve: The server encrypts the database with the shifted key, allowing the user to decrypt X_i .

Background • Quantum Key Distribution (QKD)

exchanges quantum-encoded key bits to establish a shared secret key and detect eavesdropping.



Strength: Loss-tolerant: Lost quantum states are discarded during key generation, so the user does not need to resend the same query.

Limitation: The server may infer partial query information, weakening user privacy.

Quantum privacy for databases - quantum approaches proposed

Approach 3 · QRAC-Based Quantum SPIR

1. Map data rows to measurement bases

$$\text{Row } r \leftrightarrow B_r$$

2. Encode rows into shared QRAC states

$$|\psi_\ell\rangle = \text{QRAC}(b_{1,\ell}, \dots, b_{R,\ell})$$

where $b_{r,\ell}$ is block ℓ of row r .

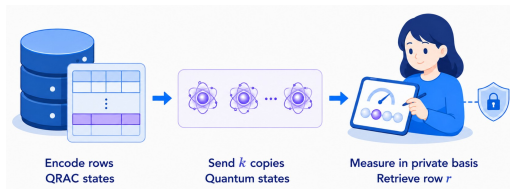
3. Send k copies and measure privately

$$|\psi_\ell\rangle^{\otimes k} \xrightarrow{\text{Measure in } B_r} b_{r,\ell}$$

The user combines the measured blocks to

Background · Quantum Random Access Code

(QRAC) encodes m classical bits into $n < m$ qubits, allowing any chosen bit to be recovered by measurement. For example, a $3 \rightarrow 1$ QRAC achieves $P_{\text{succ}} \approx 0.789$.



Strength: Embed privacy into data encoding, avoiding qRAM-based query execution (Approach 1) and full-database transmission (Approach 2).

Limitation: Retrieval is probabilistic; increasing k improves accuracy but weakens data privacy.

04

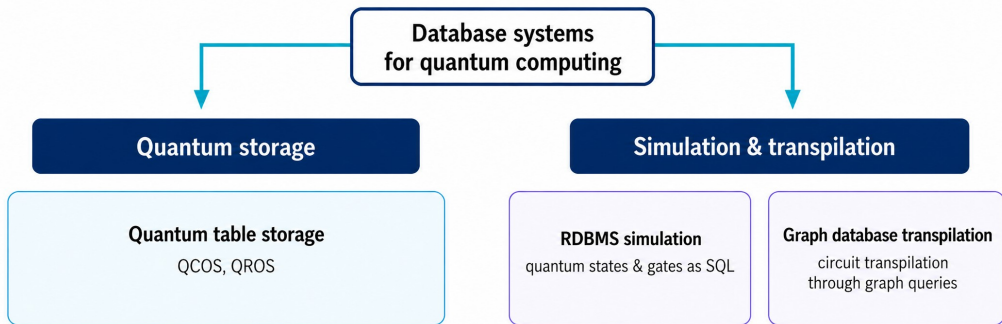
PART 4 OF 6

Data Management for Quantum Computing

Storage, simulation, and transpilation.

The landscape of data management for quantum computing

Beyond using quantum computing to accelerate database operations, database systems can leverage their own data-management capabilities to support quantum workloads.



Quantum storage design - QROS and QCOS

Motivation: Relational tables need database-oriented quantum storage formats to bridge classical RDBMS data and native quantum operations.

Example Table

RowID	A	B
0	0	1
1	1	0

Quantum Column-oriented Store (QCOS)

TableID	ColumnID	RowID	Data
0	0	0	0
0	1	0	1
0	0	1	1
0	1	1	0

Quantum Row-oriented Store (QROS)

Basis-state format: $|\text{RowID}, A, B\rangle$

Example: $|\psi_{\text{QROS}}\rangle = \frac{1}{\sqrt{2}}(|0,0,1\rangle + |1,1,0\rangle)$

One basis state = one complete row

Basis-state format:

$|\text{TableID}, \text{ColumnID}, \text{RowID}, \text{Data}\rangle$

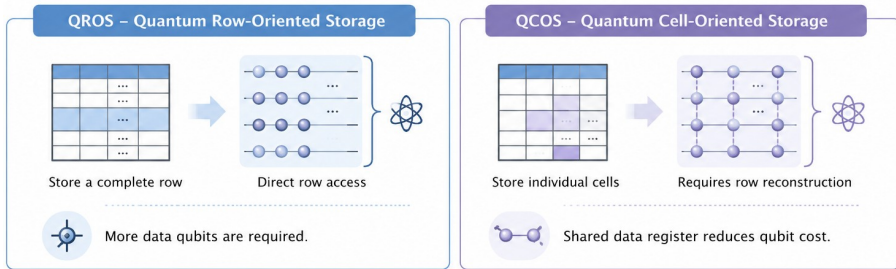
Example: $|\psi_{\text{QCOS}}\rangle = \frac{1}{2}(|0,0,0,0\rangle + |0,1,0,1\rangle + |0,0,1,1\rangle + |0,1,1,0\rangle)$

One basis state = one table cell

Quantum storage design - trade-offs and practical limitation

Different quantum storage designs make different trade-offs.

QROS stores **complete rows** for **direct access** at **higher qubit cost**,
while QCOS stores **individual cells** to **share data qubits** but needs **row reconstruction**.

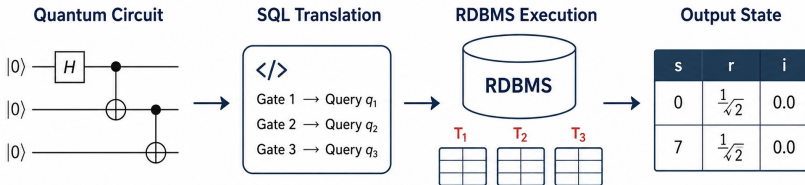


Practical limitation: measurement collapse

Measuring the quantum state collapses it. The state must be re-prepared for the next query, adding **loading overhead**.

DB for quantum simulation and transpilation

- **Motivation:** In-memory simulators are constrained by the exponential memory cost of quantum states, while database systems may leverage out-of-core processing to support larger-scale quantum simulations.



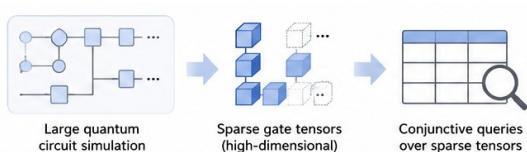
- **Main idea:** Store quantum states as relational tables and express quantum gates as SQL queries.
- **System Implementation:** **Qymera** translates quantum circuits into SQL and supports circuit construction, multiple simulation methods, and benchmarking.
- **Benefits:** Database out-of-core processing enables larger quantum simulations beyond main-memory capacity.

[1] Tim Littau and Rihan Hai. 2025. Qymera: Simulating Quantum Circuits using RDBMS. In SIGMOD Conference Companion. ACM, 179–182.

[2] Immanuel Trummer. 2024. Towards Out-of-Core Simulators for Quantum Computing. In Proceedings of the 1st Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Santiago, AA, Chile) (Q-Data '24).

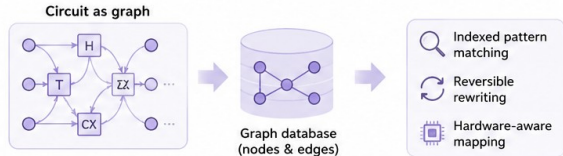
DB for quantum simulation and transpilation - new explorations

Quantum Conjunctive Queries



- **Main idea:** Translate circuit simulation into conjunctive queries over sparse gate tensors, exploiting sparsity and circuit structure to avoid unnecessary computation.
- **Limitation:** Practical performance against existing simulators remains unverified.

Transpiling Quantum Circuits on Graph Databases



- **Main idea:** Store quantum gates as nodes and their dependencies as edges in a graph database, enabling indexed pattern matching, reversible rewriting, and hardware-aware mapping.
- **Limitation:** Rewritten ZX-graphs may require sequential extraction to recover valid quantum circuits.

[1] Floris Geerts and Rihan Hai. 2025. QC meet CQ: Quantum Conjunctive Queries. In Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25). Association for Computing Machinery, New York, NY, USA, 25.

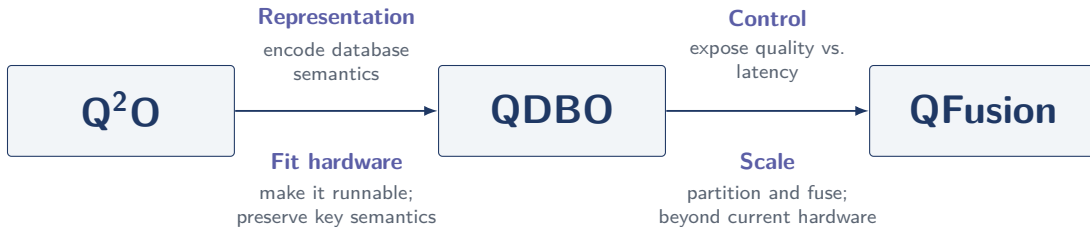
[2] Valter Johan Edvard Uotila. 2025. Transpiling Quantum Circuits on Graph Databases. In Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25). Association for Computing Machinery, New York, NY, USA, 18–19.

05

PART 5 OF 6

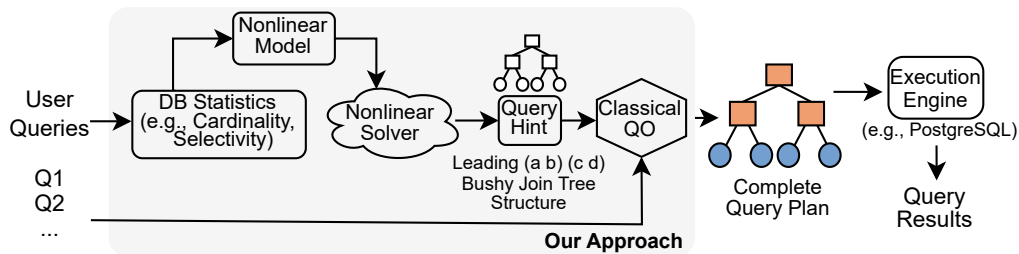
QC and DB Meet

From Algorithm to Deployed Component: A Systems Roadmap



The following systems close four gaps on the way from algorithm to deployed system.

Beyond Left-Deep: Bushy Join Trees on a Hybrid Solver

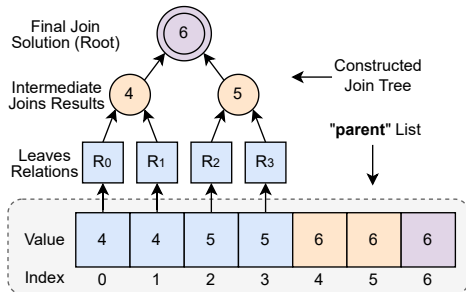


1. Construct. PostgreSQL statistics become an NL-Model.

2. Solve. The NL-Solver returns a bushy join order hint.

3. Execute. PostgreSQL completes and runs the physical plan.

Efficient Encoding for Bushy Join Trees



One integer list of $T = 2n - 1$ entries encodes any join tree.

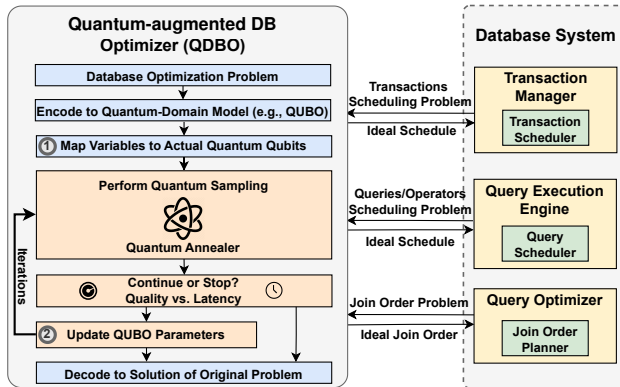
- NL-Solver, not QUBO
- 17 relations (JOB q102) versus 5 for the prior direct-QUBO bushy method
- Decoded to a Leading hint in PostgreSQL

92.7%

Largest execution-time reduction (JOB q21),
with 31 of 113 queries improved

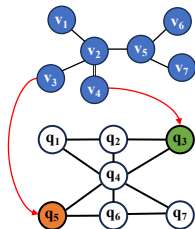
The search space grows while the encoding shrinks to one parent list.

QDBO: A White-Box, Budget-Aware Database Optimizer



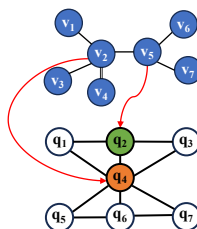
Two phases: Reduce the QUBO to fit the hardware, then use samples to correct it.

Phase 1: Reduce the QUBO to Fit the Hardware



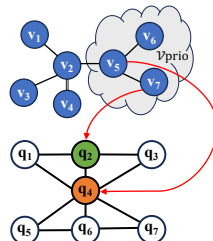
(a) Random

Lowest overhead.



(b) Greedy

Strongest connectivity.



(c) Semantic

Keeps semantic couplings.

From Variable Mapping to QUBO (Coupling) Reduction

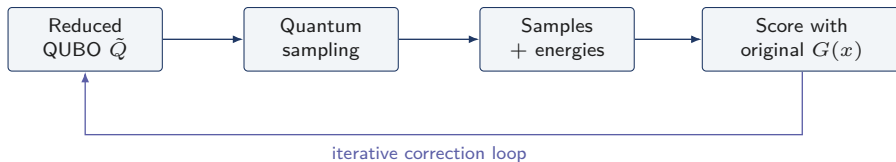
Why Can't We Use Energy Pruning to Reduce Couplings?

A small-weight coupling can still carry a **semantic constraint**; after pruning, the decoded plan may be invalid.

Phase 2: Use Samples as Feedback to Correct the Model

Core Motivation

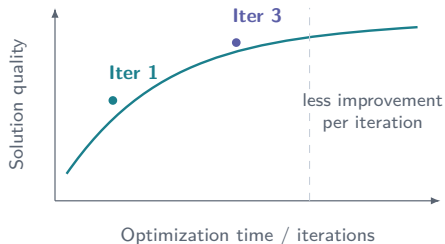
Shrink the difference between the reduced QUBO and the original database optimization problem.



$$(h^{(t+1)}, J^{(t+1)}) = (h^{(t)}, J^{(t)}) - \eta \left(\widehat{\nabla_h D_{\text{KL}}}, \widehat{\nabla_J D_{\text{KL}}} \right)$$

Sample-based gradient descent.

The Iteration Loop Exposes the Quality vs. Latency Trade-Off



Stop Condition

Iteration cap, time budget, or quality target.

Fast Path

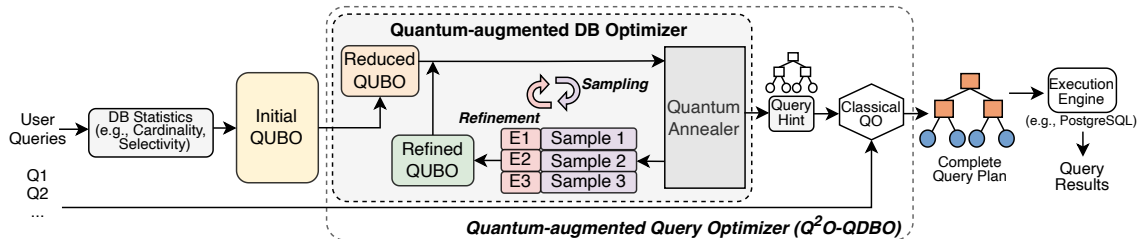
Iter 1 when planning latency dominates.

Quality Path

Keep correcting to improve the results.

The database optimizer, not the quantum service, decides how much time to spend solving.

Q²O-QDBO Integrates with PostgreSQL



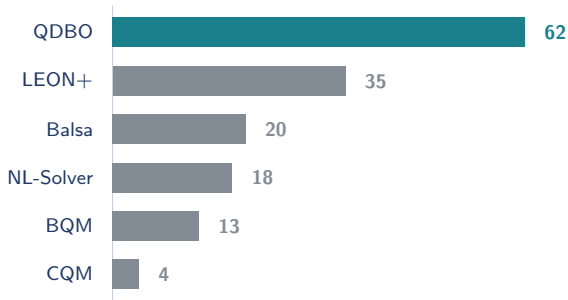
DB statistics → QUBO + mapping → sample feedback → query hint → physical plan

Judge “quantum benefit” across the complete query lifecycle, not from solver output alone.

QDBO Results: Coverage and Speedup

JOB queries improved over PostgreSQL

out of 113



Ours: Iter 3, semantic mapping

49.36×

Maximum execution speedup
CEB, Iter 3 semantic

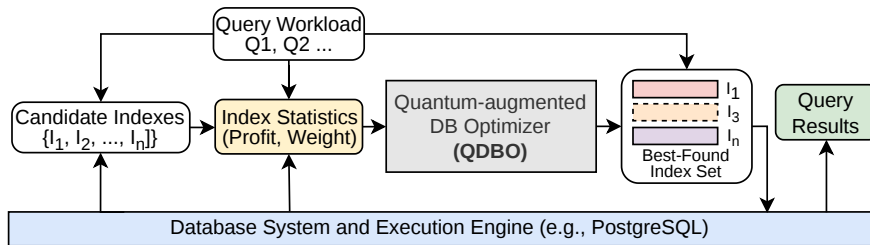
36.88×

Best end-to-end speedup
CEB, Iter 1 semantic

Iter 3 semantic reaches 34.65× end to end.

Improve both plan quality and end-to-end latency.

Generality: From Join Ordering to Index Selection

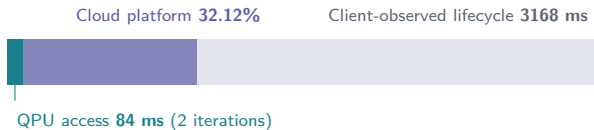


2.64×

TPC-H, empty-index baseline, $B = 50\%$, semantic mapping

QDBO is a quantum-augmented framework for broader database optimization problems.

Real Time: Where the Milliseconds Actually Go



Communication is the **major cost**.

1.78–2.65×

Three example queries, Iteration 2,
all quantum overhead included

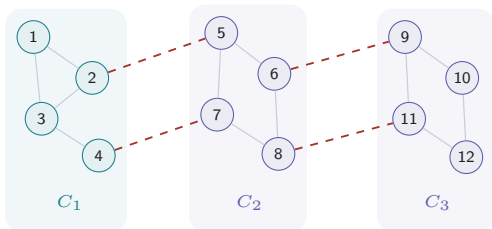
- Iter 2: 36 of 113 JOB queries improved
- q102: Up to 14.02× execution (Iter 2)
- Server Solving 508.8 ms; QPU 42.1 ms

The end-to-end bottleneck lies in system integration, not QPU access.

QFusion: Scaling Oversized Problems Beyond Hardware Limits

A 50-relation join order QUBO can exceed 10,000 variables against $\sim 5,000$ qubits, so decomposition is unavoidable.

--- cross-partition couplings



$$\min_{\mathbf{x}} E(\mathbf{x}) \longrightarrow \sum_{i=1}^m \min_{\mathbf{x}_{C_i}} E_i(\mathbf{x}_{C_i}) + \min_K E_{\text{cut}}(K)$$

Local Solving

Each component returns low-energy candidates.

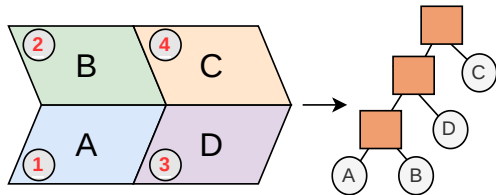
Global Fusion

Final energy depends on the fusion order of cross-couplings.

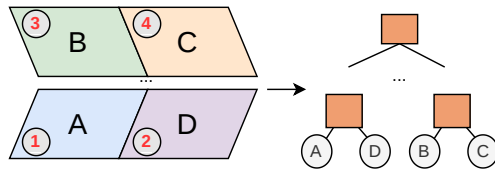
After decomposition, fusion becomes a first-class optimization problem.

A Fusion Plan Is a Binary Tree, Just Like a Join Plan

A query optimizer chooses a **join order**; QFusion chooses a **component fusion order**.



Linear Fusion Tree



Bushy Fusion Tree

QFusion Costs a Tree by Its Boundary Cuts

TREE COST

$$C(T) = \sum_{v \in \text{Internal}(T)} C_{\text{merge}}(L(v), R(v))$$

MERGE COST

$$C_{\text{merge}}(L, R) = |L \cup R| \cdot \text{cut}(L, R), \quad \text{cut}(L, R) = \sum_{u \in L, v \in R} |J_{uv}|$$

1 · Measure the cut

Sum coupling strengths across the cut.

2 · Weight by scope

Larger merge scope means higher cost.

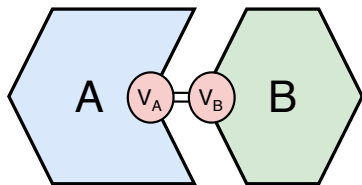
3 · Plan globally

DP selects the lowest-cost tree.

Cut structure alone prices the tree: Plan it *before* any quantum execution.

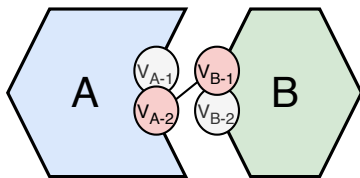
Three Merge Strategies

Boundary awareness increases from left to right.



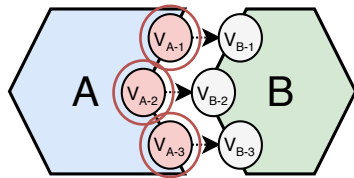
Direct Fusion

Best of each side, concatenated.



Top- k Merge

Score every pair on boundary energy,
keep the k best.

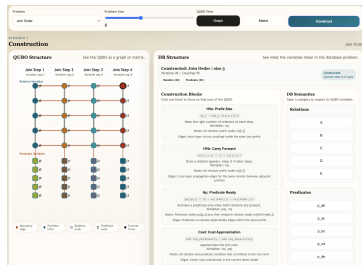


Conditioned Fusion

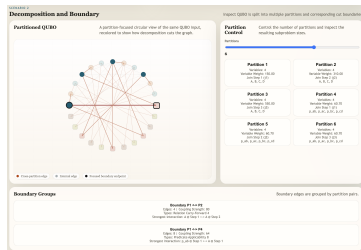
Fix the upstream choice, re-rank
downstream under it.

Expose a trade-off between solution quality and overhead.

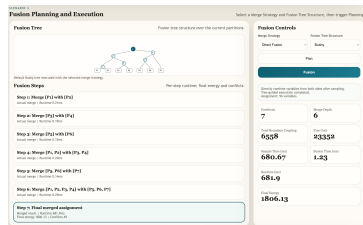
Live Demo: Plan and Execute a Fusion Tree



S1 · Construct QUBO



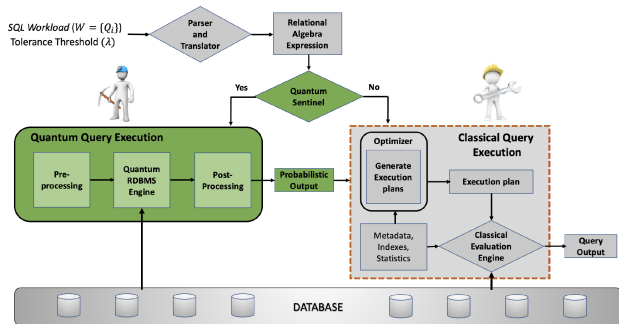
S2 · Inspect Boundaries



S3 · Plan and Execute

Live task: Choose a merge strategy, execute the fusion plan, and compare boundary cost.

A Hybrid Quantum-Classical Path for SQL Execution



Two paths, one shared interface: both routes return ordinary tuples, but only the hybrid route accepts a declared false-negative tolerance λ .

1. Route the query. The Quantum Sentinel chooses a classical or hybrid execution path.

2. Sample candidates. Each quantum measurement samples a candidate row ID.

3. Validate quickly. The DBMS verifies those candidates with index lookups.

The quantum path narrows the search; the DBMS turns probabilistic candidates into verified tuples.

A Database Lifecycle under Quantum Rules

How do familiar database operations act on one shared quantum state?

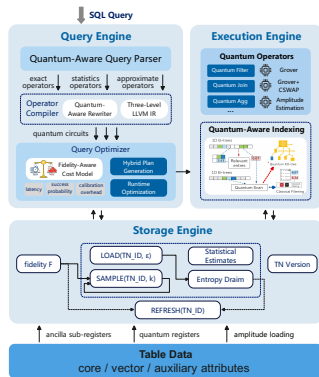


What is an entry? An index is paired with one data value inside the shared quantum state.

What changes? Read-out collapses the state to one entry. Extend and Remove rebalance the shared state.

Store database entries in one shared quantum state; then update or read that state through familiar operations.

Qute: A Quantum-Native Database Architecture



1. **Compile SQL.** Use a three-level IR for eligible operators.

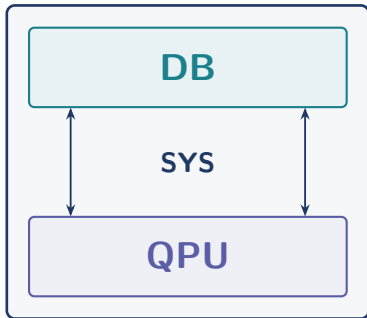
2. **Choose a hybrid plan.** Compare classical and quantum operator costs under a qubit budget.

3. **Execute selectively.** Let classical indexes prune before quantum operators run.

4. **Manage quantum state.** Keep quantum-active state separate from classical metadata.

Qute co-designs SQL compilation, hybrid planning, selective execution, indexing, and storage.

Practical Quantum Database Systems: Three Layers of Control



Semantic control

Faithful database problem formulation, query plans, and workload metrics.

Runtime control

Budgets, feedback, fusion plans, fallbacks, and end-to-end evaluation.

Hardware control

Mapping, reduction, decomposition, sampling.

The frontier is algorithm, hardware, and system co-design.

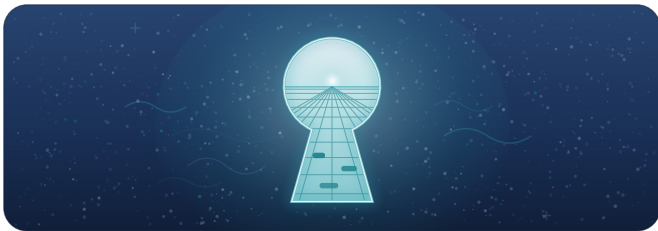
06

PART 6 OF 6

Open Challenges and Research Opportunities

From today's demonstrations to tomorrow's deployments.

What Stands between Demos and Deployment?



Demonstrated Today

Deployable Tomorrow

Q1: Representation & Execution

Q2: Controllable Algorithms

Q3: Scalability

Q4: Data Management

Our database community can contribute to all four questions.

Q1: Practical Representation and Execution

1. Representation: How Is Data Encoded?

Basis encoding: IDs and keys become bit strings. Row ID 5 $\rightarrow$ binary 101 $\rightarrow |101\rangle$; wider values need more qubits.

Amplitude encoding: numbers become state weights. A normalized two-feature row $(0.6, 0.8) \rightarrow 0.6|0\rangle + 0.8|1\rangle$. Compact for vectors, but loading and readout add cost.

For a DBMS: use bit strings for IDs and state weights for numeric vectors; count loading and readout in the query cost.

2. Execution: What Gets Harder?

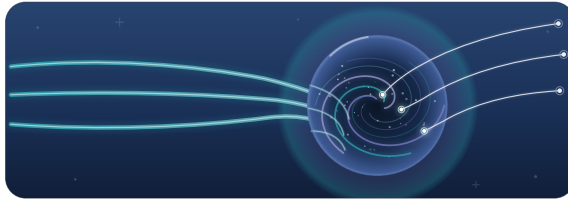
Filtering: a small real-hardware prototype. Quite runs a small filter; loading the data and checking the result remain classical.

Joins: more quantum work. Matching rows from two relations needs extra qubits, controlled operations, and deeper circuits.

Full queries: costs compound. More tables and predicates add depth and noise; comparable full-plan hardware evidence is still missing.

Encoding the data and executing the query must be costed together.

Q2: Controllable Algorithms for a Real-Time Budget



The Black-Box Problem

- No transparency into the hybrid loop
- Hybrid solver enforces a 5 s minimum response

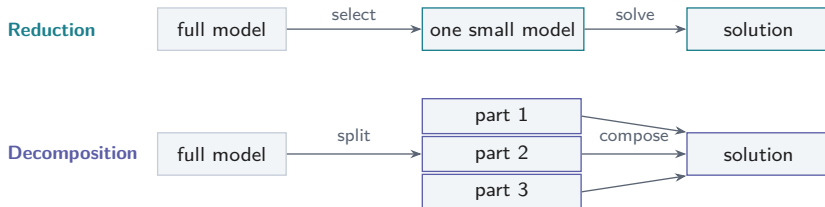
The Controllable Direction

- Control the execution flow directly
- Work inside a changeable time budget

Q3: Scalability Is a Global-Consistency Problem

Real workloads outgrow any current annealer

A 50-relation join already needs over 10,000 variables, and hardware offers about 5,000 qubits



Research Opportunity

Final quality depends on boundary handling during composition, not just on local solutions.

Q4: Reuse Data from Quantum Workflows

A run creates data worth saving: a problem split, a record of which variable used which qubit, and partial results.

1. What to save

subproblems after splitting
which variable used which qubit
partial results

2. Database operations

update after a run
retrieve for a later run
reuse when compatible

3. Who uses it?

scheduler loads saved
preparation
optimizer reuses a prior split or
mapping

Treat intermediate quantum-workflow data like reusable database records, not temporary files.

Outlook: From Quantum Prototypes to Useful Data Systems

A better future makes quantum workflows practical, scalable, and manageable.

PRACTICAL

- real workloads
- explicit latency budgets
- win in end-to-end latency

SCALABLE

- larger problem instances
- global consistency
- quality-aware composition

MANAGEABLE

- saved intermediate data
- efficient update and retrieval
- reuse when compatible

Database principles can turn isolated quantum experiments into useful, repeatable workflows.

The Two Communities Meet by Redesigning the Boundary

FORMULATE

database semantics that fit
quantum representations

ORCHESTRATE

quantum and classical
computation under latency
budgets

EVALUATE

real systems, strong baselines,
end-to-end value

Quantum Computing × **Database Systems**

Thank you

Questions and discussion

